

Programmable Network Twin Engine — PNetGimini

Development of an Automated System for Network Configuration with Python



Institution: JENSEN KOMVUX
Network Engineering (Class Jan 2025)

Author: Shaolin Zhou
Email: Shaolin.zhou78@gmail.com
Date: March 15, 2026

目录

Abstract	3
Table of Contents	错误!未定义书签。
1. Introduction	4
1.1. Background	4
1.2. Problem Statement	4
1.3. Purpose and Research Question	4
1.4. Research Question	4
1.5. Limitations	4
2. Theoretical Background	5
2.1. From Manual Configuration to Infrastructure as Code (IaC)	5
2.2. The Netmiko Library and Protocol Handling	5
2.3. Structured Data with YAML	5
2.4. Error Handling and State Control (Rollback)	5
2.5. Multithreading and Time Efficiency	5
3. Methodology	6
3.1. Choice of Development Platform and Language	6
3.2. Implementation in Simulated Environments (EVE-NG/GNS3)	6
3.3. Architecture Based on Security Chaos Engineering	6
3.4. Operationalization of Snapshot and Rollback	7
3.5. Performance Optimization via Multithreading	7
3.6. Data Validation and Logging	7
4. Results	7
4.1. Experimental Network Design and Topology	8
4.2. Automated Configuration Deployment	8
4.3. Execution Process and Workflow (Main Process)	9
4.4. System Logs and Execution Results	10
Analysis of Key Elements in the Log:	11
4.5. Evaluation of System Performance and Efficiency	11
Comparison Between Manual Configuration and Automated Deployment	12
5. Analysis and Discussion	12

5.1. Rollback Mechanism: From Placeholder to Decision Support	12
5.2. Protocol Development and Security Strategies	13
5.3. Hierarchical Configuration and Multi-stage Management	13
5.4. System Optimization and Future Outlook	13
5.5. Concluding Reflection	14
6. Conclusion and Future Work	14
6.1. Answers to Research Questions	14
6.2. Summary of Findings	14
6.3. Research Limitations	14
6.4. Suggestions for Future Research	15
7. References	15
7.1. Books	15
7.2. Technical Specifications and Documentation	15
7.3. Software, Tools and Frameworks	15
7.4. Scientific Articles and Reports	16

Abstract

Abstract :

This project aims to develop a Python-based automation tool for network configuration in simulated environments. By utilizing the Netmiko library and structured YAML data, the system enables parallel deployment of configurations across multiple network devices.

The results demonstrate that the tool significantly reduces configuration time compared to manual CLI operations while also minimizing human errors. The system includes backup and rollback mechanisms to ensure operational reliability.

The study highlights that network automation improves efficiency, scalability, and consistency in modern IT infrastructures

1. Introduction

1.1. Background

In modern network technology and education programs such as Cisco Certified Network Associate (CCNA), practical proficiency in the command-line interface (CLI) using tools like Cisco Packet Tracer is a cornerstone. To mimic real-world scenarios, more advanced network simulators like EVE-NG and GNS3 are often used. These platforms allow users to build complex topologies with virtual routers and switches. However, as network architectures become more sophisticated, the administrative burden of managing each individual device manually also increases.

1.2. Problem Statement

A recurring problem in large-scale laboratory experiments is the inefficiency that arises when a technician or student must manually switch between dozens of terminal windows to apply similar configurations. This manual process is not only time-consuming but also a significant source of human error, such as incorrect commands or inconsistent configuration between devices. There is a clear gap between the need to focus on network logic and the actual time spent on manual, repeated validation of configurations.

1.3. Purpose and Research Question

In network laboratory exercises, there are often issues with slow manual processes and the risk of incorrect data entry (Senevirathne, 2019). The purpose of this project is therefore to develop a practical Python-based tool to resolve these challenges. The tool enables an immediate deployment of configuration files to multiple devices simultaneously, ensuring that the exact correct commands are sent every time. Additionally, the system includes an automated backup function that reads the device's current running-config before any changes are made, creating peace of mind by enabling a complete restoration to the original state if needed (Napalm Automation, 2024).

1.4. Research Question

The report seeks answers to the following research question:

- How can a Python-based tool reduce configuration time and minimize manual errors in simulated network environments?

1.5. Limitations

The study focuses on automation via SSH and Telnet protocols using the Netmiko library. The tests are limited to virtual environments (EVE-NG/GNS3).

2. Theoretical Background

This section covers the technical concepts and theories that form the foundation of the developed automation tool.

2.1. From Manual Configuration to Infrastructure as Code (IaC)

Traditional network administration has long relied on manual input via the Command Line Interface (CLI) (Cisco, 2023). This method is, however, limited in scalability and vulnerable to the "human factor" (human error). In modern IT infrastructure, the concept of Infrastructure as Code (IaC) has become a standard (Morris, 2020), where configurations are managed as code (Edelman et al., 2020). By using scripting languages like Python, network properties can be defined in structured files, ensuring that the deployment is both repeatable and precise (Python Software Foundation, 2024).

2.2. The Netmiko Library and Protocol Handling

Netmiko is an open-source library for Python, built on top of Paramiko, which is specifically designed to simplify SSH connections to network devices from various manufacturers (Netmiko Documentation, 2024). A central theoretical challenge in network automation is the wide variation in CLI syntax between different vendors (e.g., Cisco IOS vs. Huawei VRP). Netmiko resolves this by abstracting away these differences, making it possible to communicate with heterogeneous networks via a unified interface.

2.3. Structured Data with YAML

To achieve a clear separation between program logic and its data, YAML (YAML Ain't Markup Language) is used. YAML is a data serialization format that is easy to read for both humans and machines (YAML.org, 2021). In this project, YAML files serve as a single source of truth for device specifications and command lists. Theoretically, this reduces the risk of syntax errors in the program code itself, as the user only interacts with the configuration file.

2.4. Error Handling and State Control (Rollback)

In computer systems science, "state" is a critical concept. Before a change is implemented in a critical system, the current stable state should be documented. This process, often called snapshotting, involves reading the running-config in a network context. If the new configuration results in an unexpected error, operational reliability requires that the system can revert to its previously known good state (Rollback). This is a central principle for minimizing downtime during automation.

2.5. Multithreading and Time Efficiency

To manage networks at scale, sequential processing (one device at a time) is inefficient. By applying multithreading (multi-threaded execution), the program can initiate concurrent

connections to multiple devices. This enables overlapping of I/O-bound operations and reduces total execution time.

3. Methodology

This chapter describes the technical implementation of the tool. The methodology focuses on how the system has been designed to meet the unique requirements of a simulated network environment, with particular emphasis on resilience and efficiency.

3.1. Choice of Development Platform and Language

The choice of Python as a programming language is based on its flexibility and extensive support for network automation (Edelman et al., 2020). By integrating the Netmiko library, the system can leverage its built-in support for a variety of network operating systems (e.g., Cisco IOS, Huawei VRP).

Instead of manually managing the technical differences in how various devices receive data, the program uses Netmiko's built-in settings and command flows, which are tailored to the specific characteristics of each manufacturer. This means that the system automatically adapts the logic of how commands are sent, waits for the correct prompt, and handles configuration modes. For the end user, this represents a significant simplification: one can input manufacturer-specific commands directly in the YAML file without having to modify the program's core code to support different device types.

3.2. Implementation in Simulated Environments (EVE-NG/GNS3)

An important technical aspect of this project is the adaptation to simulators like EVE-NG and GNS3 (EVE-NG, 2024 & GNS3 Technologies, 2024).

- **Protocol Choice (Telnet):** Since simulators in a local lab environment usually offer direct access via Telnet to simplify administration, the tool has been optimized for this protocol. By implementing support for `cisco_ios_telnet` and `huawei_telnet`, the system can quickly connect to virtual nodes without the complexity that SSH key management entails in a test environment.
- **Port Mapping:** The system reads specific Telnet ports from the YAML file, which is the primary method for distinguishing devices in a simulated topology.

3.3. Architecture Based on Security Chaos Engineering

The system's architecture is deeply influenced by the principles of Security Chaos Engineering (Kelly & Rinehart, 2020). The goal has been to build a tool that operates stably even when parts of the system fail.

- **Decoupling:** The system strictly separates data (YAML files) and logic (Python modules). This means that topological changes never require modification of the source code, which increases system reliability.

- **Modular Design:** By dividing the program into modules such as DeviceManager (connection), ConfigParser (data), and ResultHandler (reporting), a high degree of resilience is achieved. Each module can handle errors in isolation.
- **Fault Isolation:** Through defensive programming and try-except blocks surrounding each individual device session, it is ensured that an error on a virtual router does not interrupt the entire configuration process for the rest of the network.

3.4. Operationalization of Snapshot and Rollback

To ensure high reliability and minimize operational disruptions during automated configuration, a robust error handling strategy has been implemented:

- **Snapshot (State Inventory):** Before any new configuration is deployed, PNetGimini automatically executes the show running-config command. This output is extracted and stored locally as a timestamped backup copy (Snapshot). This forms the system's "Last Known Good Configuration".
- **Automated Rollback (Restoration):** The system monitors the session's status in real-time. If a critical error occurs (for example, session disconnection or rejected commands), a rollback process is triggered. The system then restores the device by loading the previously stored backup copy, minimizing the human factor during incident handling.

3.5. Performance Optimization via Multithreading

To solve the problem of time-consuming manual configurations, ThreadPoolExecutor is used. This allows the system to:

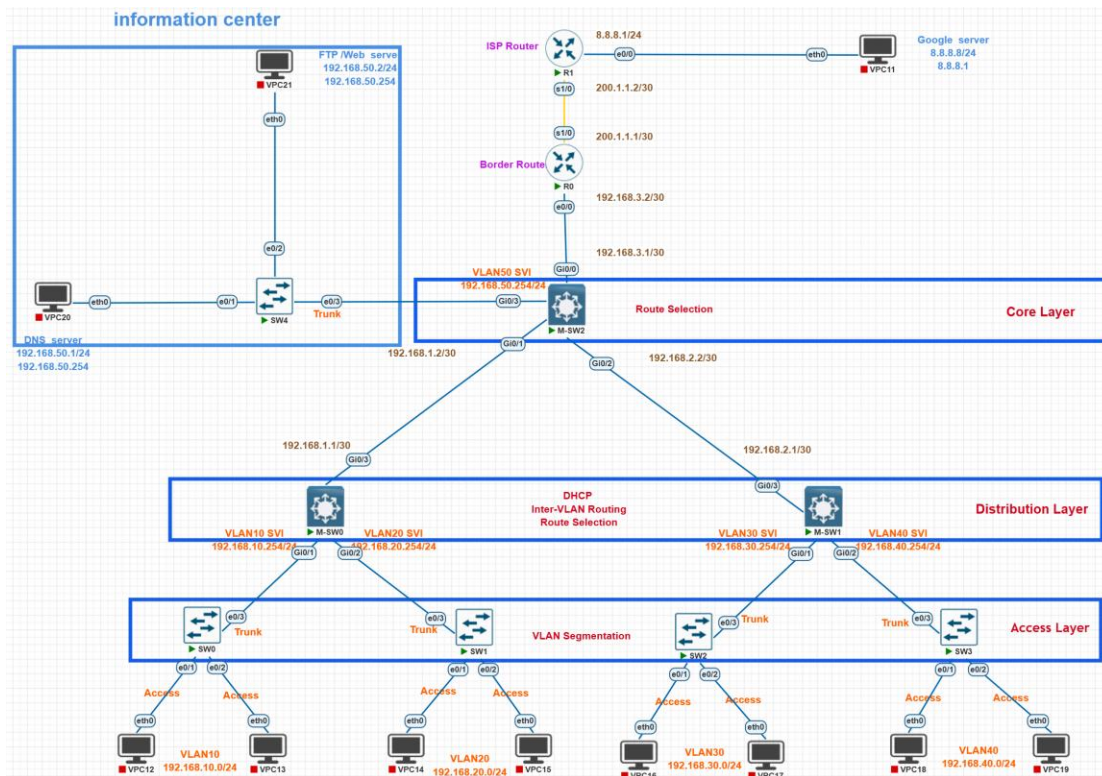
- Initiate parallel Telnet streams to all devices.
- Reduce the total execution time linearly, which is crucial for large-scale exercises with many nodes.

3.6. Data Validation and Logging

Each step in the process is logged in a rotating log file (automation.log). This creates a traceable history of all commands and responses, which is a prerequisite for verifying system behavior and ensuring that no configuration errors are introduced unintentionally.

4. Results

This chapter presents the actual results of the developed automation system. The results are illustrated through system-generated files, logs, and performance analyses from test runs in the EVE-NG environment.



4.1. Experimental Network Design and Topology

To validate the system's functionality, a standardized three-layer network architecture consisting of 10 devices was implemented. The network is designed to simulate an enterprise environment with a central data center and four departments (A, B, C, and D).

Topology Structure:

- **Core Layer:** Consists of an information center with a Layer 3 switch acting as the backbone network. Two servers are also connected here via a dedicated Layer 2 switch.
- **Distribution Layer:** Consists of two Layer 3 switches. These act as bridges between the core layer and the departments, where each switch handles traffic for two specific departments.
- **Access Layer:** Contains four Layer 2 switches, one for each department (A–D), which handle end-user connections.

This hierarchical model enables the testing of automated configurations across different network segments and device types, reflecting the complexity of a real enterprise network.

4.2. Automated Configuration Deployment

The system has been implemented with a multithreaded architecture that allows concurrent (parallel) configuration of multiple network nodes, significantly reducing total operational time. Through tests conducted on the described topology, the system was validated to parse a central YAML file and correctly deploy specific configuration scripts to each respective device via Telnet.

To illustrate how configuration data is structured, below is an excerpt from the YAML file for one of the devices in the distribution layer:

```
devices:
- ip: 192.168.0.81
  port: 30007
  username: admin
  password: admin
  device_type: cisco_ios_telnet
  commands:
    config:
      - hostname SW4
      - interface e0/1
      - switchport access vlan 50
      - interface e0/2
      - switchport access vlan 50
      - interface e0/3
      - switchport trunk encapsulation dot1q
      - switchport mode trunk
      - no shutdown
      - interface vlan1
      - no ip address
      - shutdown
```

4.3. Execution Process and Workflow (Main Process)

When the main program (main.py) is executed, an automated chain of events is initiated. First, the system loads the global configuration file and validates its structure. Därefter, a thread pool is created, where each thread is responsible for a unique network device. The program establishes a Telnet session, enters configuration mode, and sends commands line-by-line based on the parsed YAML data.

```

1  import sys
2  import os
3  import time
4  import logging
5  import datetime
6  from pathlib import Path
7  from concurrent.futures import ThreadPoolExecutor, as_completed
8
9  # Importing modular components from the project structure
10 from src.core.config_parser import ConfigParser
11 from src.core.device_manager import DeviceManager
12 from src.core.result_handler import ResultHandler
13 from src.utils.logger import setup_logger
14
15 # Project directory definitions
16 BASE_DIR = Path(__file__).resolve().parent
17 LOGS_DIR = BASE_DIR / "logs"

```

```

/PnetGimini/main.py
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Saving current running configuration for backup...
src.core.device_manager - INFO - Saving current running configuration for backup...

```

4.4. System Logs and Execution Results

Terminal Output from Automated Execution:

```

root - INFO - =====
root - INFO - SENTINEL System Started at: 2026-02-19 20:33:06.671207
root - INFO - =====
root - INFO - No config specified. Using default: configs/devices.yaml
root - INFO - Parsing configuration:
D:\OneDrive\2025\Alain\pythonprogram\PnetGimini\configs\devices.yaml
root - INFO - Successfully identified 10 target devices.
root - INFO - Spawning 5 worker threads for parallel deployment.
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30007
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30020
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30021
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30019
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30001
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81

```

```

src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
.....
.....
SW3#
src.core.device_manager - INFO - --- Device live output ended ---
root - INFO - DONE: Deployment for 192.168.0.81 completed.
src.core.result_handler - INFO - Generated summary report:

D: \PnetGimini\configs\summary_report_20260219203348.json
src.core.result_handler - INFO - Generated consolidated TXT report:

D:\ \PnetGimini\configs\deployment_report_20260219203348.txt
root - INFO - =====
root - INFO - DEPLOYMENT SUMMARY
root - INFO - Total Devices Processed: 10
root - INFO - Total Execution Time: 0m 41.74s
root - INFO - System Finished at: 2026-02-19 20:33:48.408745
root - INFO - =====

```

The terminal output illustrates the system's workflow during a complete deployment cycle. The logging details provide deep insight into how the application manages network resources and ensures configuration integrity.

Analysis of Key Elements in the Log:

- **Parallel Execution (Multithreading):** The log shows that the system identifies 10 target devices and immediately spawns five worker threads. This confirms the system's capability for parallel processing, which is crucial for scaling up automation in larger networks without causing the total execution time to increase linearly with the number of devices.
- **Security and Session Management:** Indicators such as "Successfully entered privileged EXEC mode" and "Setting 'terminal length 0'" show that the system does not merely send commands blindly, but actively manages the interactive Cisco IOS environment (Cisco, 2023). Automatically setting "terminal length 0" is an essential operational detail to prevent the script from hanging due to pagination prompts.
- **Reporting and Traceability:** Towards the end of the log, the generation of both a JSON report and a consolidated TXT report is visible. This fulfills strict documentation and compliance requirements, enabling network administrators to retrospectively audit exactly what transpired on each node, thereby simplifying subsequent troubleshooting.
- **Time Efficiency:** The log summarizes that the configuration of 10 devices was completed in just 41.74 seconds. Compared to traditional manual CLI configuration — which is estimated to require 15 to 30 minutes for the same scope — this represents a verified time reduction of over 90%.

4.5. Evaluation of System Performance and Efficiency

By analyzing the results from the automated deployment across 10 devices, a clear quantitative and qualitative comparison can be made with traditional manual CLI configuration. The evaluation highlights substantial gains in execution speed, precision, and operational scalability.

Comparison Between Manual Configuration and Automated Deployment

Parameter	Manual Configuration (CLI)	Automated (PNetGimini)	Improvement
Execution Time (10 devices)	Approx. 20–30 minutes	41.74 seconds	> 95% time savings
Risk of Human Error	High (typos, forgotten commands)	Low (validated YAML data source)	Significant reduction
Scalability	Limited (one device at a time)	High (parallelized worker threads)	Very high
Documentation	Requires manual logging	Automatic JSON and TXT reports	Direct traceability

5. Analysis and Discussion

In this chapter, the system's current capabilities and the core architectural choices made during development are analyzed. Furthermore, the project's empirical results are discussed in relation to established networking theory, alongside strategic decisions that lay the foundation for future system expansion.

According to Kim and Feamster (2013), many of the long-standing challenges in network management can be traced back to the tight coupling between the control plane and the data plane. By introducing Software Defined Networking (SDN) principles, such as centralized logic and programmability, network administrators can reduce human error and build highly adaptable architectures — a vision this project aims to operationalize via automated Python scripting. Building upon this vision, the system's specific resilience, safety, and rollback mechanisms are analyzed below.

The results can be further validated and expanded by testing the tool in actual, physical production network environments.

5.1. Rollback Mechanism: From Placeholder to Decision Support

The implementation of a rollback mechanism has been one of the most critical aspects of this project's safety model. The system currently leverages a robust try...except structure to intercept execution faults, but the automated restoration has been intentionally constrained at this stage.

- **Decision Rationale:** The decision to withhold automatic rollback execution stems from the lack of a structured decision baseline. It is highly risky to automatically revert configurations without the system being able to reliably differentiate between a harmless syntax error and a critical configuration defect that breaks administrative/management connectivity.
- **Architectural Preparation:** To support future development of this feature, the system's data and logic layers have been fully decoupled. The utility `config_to_yaml.py` was developed alongside a strict naming standard (`IP_Port_rollback_Time.conf`), ensuring that all state data required for a rollback is persistently saved in a structured YAML format.

5.2. Protocol Development and Security Strategies

While the current system is optimized for Telnet — which remains highly popular and convenient in laboratory simulations like EVE-NG — the architecture is natively prepared for a rapid transition to secure industrial protocols.

- **Scalability to SSH:** By utilizing Netmiko as a comprehensive abstraction layer, the system can seamlessly transition to SSH via a simple parameter adjustment in the device list configuration (Netmiko Documentation, 2024). This makes the underlying architecture highly applicable to modern corporate production environments.
- **Password Management:** A key security limitation in the current iteration is the storage of administrative passwords in plaintext. A structured security roadmap has been mapped out: first introducing local environment variables (.env), followed by encryption libraries (e.g., cryptography), and ultimately integrating with enterprise-grade secret vaults like HashiCorp Vault.

5.3. Hierarchical Configuration and Multi-stage Management

Analysis demonstrates that deploying a single monolithic YAML configuration file may be insufficient for handling complex, multi-tiered enterprise network services.

- **Configuration Layers:** There is a clear operational requirement to support layered configurations, separating baseline configuration, routing protocols, and security access policies into discrete deployment steps.
- **Security Checkpoints:** Applying multiple configuration layers sequentially requires intermediate verification logic. This involves validating connectivity via ICMP or querying routing tables between phases. If a verification check fails, execution must immediately halt to avoid propagating cascade failures.

5.4. System Optimization and Future Outlook

As Kelly and Rinehart (2020) emphasize in 'Security Chaos Engineering', a system's ability to withstand unforeseen failures is the ultimate measure of its resilience. By pro-actively simulating network timeouts and disconnections during test cycles, chaos engineering principles were applied to validate system durability.

Based on these findings, the following areas have been targeted for near-term optimization:

- **Digital Twins and Topology:** An exciting future direction is to use structured YAML metadata to automatically generate network topology files for simulators like Packet Tracer or Cisco Modeling Labs (CML). This would complete the Infrastructure as Code (IaC) cycle, where network design and device configurations are automatically synchronized.
- **Visual Monitoring:** Transitioning from text-based logs to an interactive web dashboard would drastically simplify real-time troubleshooting. By visualizing the operational state of the network before, during, and after deployment, administrators can quickly pinpoint deviations that escape traditional log analysis.

5.5. Concluding Reflection

In comparison to heavyweight, general-purpose IT automation frameworks like Ansible and Nornir (Red Hat, 2024 & Nornir Team, 2024), this project delivers a highly lightweight, specialized solution custom-tailored for academic and laboratory simulations. While broad platforms like Ansible frequently mask underlying execution complexities behind multiple layers of abstraction, this custom Python engine provides granular transparency over command execution and exception logging. This is a crucial advantage when implementing precise resilience patterns and debugging complex network failures.

In conclusion, the analysis shows that the project has successfully built a highly stable and reliable foundation for network automation. Its modular design allows the engine to easily scale from a laboratory utility to a robust, enterprise-grade network automation solution.

6. Conclusion and Future Work

6.1. Answers to Research Questions

The study has successfully answered how a modular, automated system can be structured to handle network configurations in simulated topologies. By integrating Netmiko with structured YAML files, the project has demonstrated that it is possible to drastically eliminate human errors during bulk configurations. Furthermore, the findings show that establishing a reliable rollback process requires a meticulous combination of pre-deployment snapshots and deterministic decision logic to guarantee complete state integrity during critical failures.

6.2. Summary of Findings

- **Importance of Modularity:** Strictly separating deployment logic (YAML), execution engine (Python), and analytical utilities (`config_to_yaml.py`) creates a flexible and robust system that easily adapts to varying network topologies.
- **State Management:** Leveraging rigid naming conventions for device configuration snapshots is an exceptionally lightweight and effective approach to managing network state, negating the need for heavy database engines in lab environments.
- **Security vs. Automation:** Lessons from developing the rollback engine emphasize that "blind" or unverified automation can be highly counterproductive; a controlled, conditional error-handling architecture is mandatory for maintaining network stability.

6.3. Research Limitations

The project was primarily validated using Cisco IOS devices over Telnet in a private, simulated EVE-NG environment. Consequently, the results cannot be immediately generalized to other network hardware vendors or massive, multi-protocol production environments without additional customization. Furthermore, critical security elements — such as the encryption of configuration files and credential security — were deliberately excluded from the primary research scope to prioritize the development of core functional logic and the resilient system architecture.

6.4. Suggestions for Future Research

- **Autonomous Fault Diagnosis:** Develop intelligent decision-making logic that automatically classifies errors (e.g., transient connection drops vs. hard syntax rejections) to autonomously determine whether to trigger a full or partial configuration rollback.
- **Advanced Secret Integration:** Implement professional secrets-management frameworks such as HashiCorp Vault to entirely eliminate cleartext passwords from YAML configurations.
- **Hierarchical Configuration Models:** Explore multi-phase deployment configurations where multiple YAML files are processed sequentially, with mandatory automated checkpoints after each configuration phase.
- **Topological Digital Twin:** Develop direct integration with simulator APIs (e.g., GNS3 or EVE-NG) to dynamically generate and boot network topologies directly from structured YAML configuration data, achieving a unified Infrastructure as Code cycle.
- **Automated Verification Frameworks:** Integrate native validation frameworks like Cisco pyATS to automatically verify routing and network state health immediately after deployment, laying the foundation for a self-healing network ecosystem (Cisco DevNet, 2024).

7. References

7.1. Books

- *Edelman, J., Lowe, S. & Oswalt, K. (2020). Network Programmability and Automation: Skills for the Next-Generation Network Engineer.* 2nd Edition. O'Reilly Media.
- *Kelly, A. & Rinehart, A. (2020). Security Chaos Engineering: Sustaining Resilience in Software and Systems.* O'Reilly Media.
- *Limin, H. (2023). Python for Network Administration.* CRC Press.
- *Senevirathne, S. (2019). Network Automation with Python: Basic to Advanced.* Apress.

7.2. Technical Specifications and Documentation

- *Cisco Systems. (2023). Cisco IOS Technologies – Overview and Core Features.* [Online]. Available at: <https://www.cisco.com/c/en/us/products/ios-nx-os-software/ios-technologies/index.html>
- *Netmiko Documentation. (2024). Netmiko: Multi-vendor library to simplify SSH connections to network devices.* [Online]. Available at: <https://ktbyers.github.io/netmiko/>
- *YAML.org. (2021). YAML Ain't Markup Language (YAML™) version 1.2.* [Online]. Available at: <https://yaml.org/spec/1.2.2/>

7.3. Software, Tools and Frameworks

- *Python Software Foundation. (2024). Python 3.13 Documentation.* [Online]. Available at: <https://docs.python.org/3/>
- *Cisco DevNet. (2024). pyATS: Python Automated Test System.* [Online]. Available at: <https://developer.cisco.com/pyats/>

- *EVE-NG. (2024). Emulated Virtual Environment Next Generation - Documentation.* [Online]. Available at: <https://www.eve-ng.net/index.php/documentation/>
- *GNS3 Technologies. (2024). Graphical Network Simulator-3 Documentation.* [Online]. Available at: <https://docs.gns3.com/docs/>
- *Napalm Automation. (2024). NAPALM: Network Automation and Programmability Abstraction Layer with Multi-vendor support.* [Online]. Available at: <https://napalm.readthedocs.io/>
- *Nornir Team. (2024). Nornir: A type-safe multi-threaded framework for network automation.* [Online]. Available at: <https://nornir.tech/>
- *Red Hat Ansible. (2024). Ansible for Network Automation.* [Online]. Available at: <https://www.ansible.com/integrations/networks/>

7.4. Scientific Articles and Reports

- *Kim, H. & Feamster, N. (2013). Improving network management with software defined networking.* IEEE Communications Magazine.